

Operating Systems Assignment 01

Custom Kernel Compilation and System Call Implementation

Roll Number: K240707

March 20, 2026

Contents

1	Prerequisites and Dependency Installation	2
2	Downloading and Extracting the Kernel Source	2
3	Makefile Modification (Kernel Branding)	3
4	System Call Implementation	3
4.1	Creating the System Call Logic	4
4.2	Linking the System Call	4
4.3	Updating the System Call Table	4
4.4	Updating the Header File	4
5	Kernel Configuration	6
6	Bypassing Security Certificates	6
7	Final Compilation	7
8	Kernel Installation and System Reboot	7
9	Verification	8
9.1	Verifying the Kernel Version	8
9.2	Writing the Test Program (<code>task.c</code>)	8
9.3	Executing and Checking Logs	9

1 Prerequisites and Dependency Installation

Before beginning the kernel compilation process, it is essential to install the necessary dependencies. This ensures that the system has all the required compilers, libraries, and tools to build the kernel from source without failing midway.

```
1 sudo apt update
2 sudo apt install build-essential libncurses-dev bison flex libssl
  -dev libelf-dev -y
```

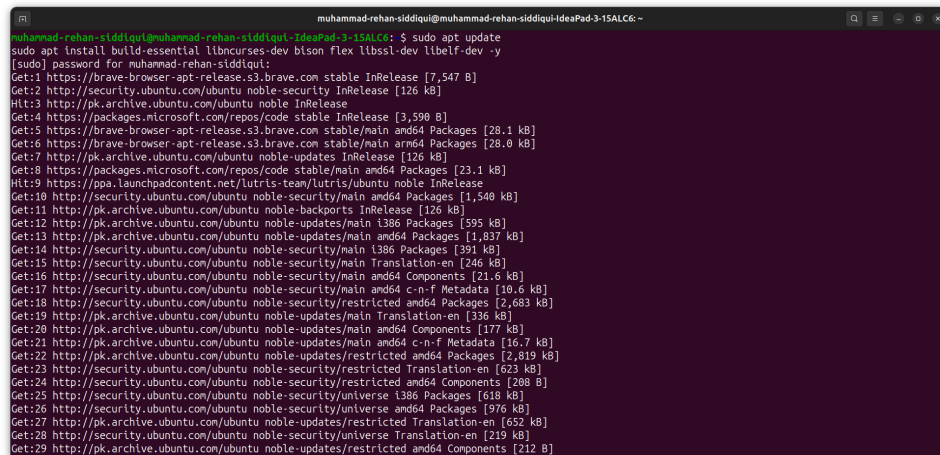


Figure 1: Installing system dependencies

2 Downloading and Extracting the Kernel Source

The next step is to acquire the raw Linux kernel source code. I downloaded kernel version 6.1.80 directly from the official Linux kernel archive and extracted the tarball into my local directory.

```
1 wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.80.
  tar.xz
2 tar -xvf linux-6.1.80.tar.xz
3 cd linux-6.1.80
```

```

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.80.tar.xz
tar -xvf linux-6.1.80.tar.xz
cd linux-6.1.80

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80
linux-6.1.80/scripts/ld-version.sh
linux-6.1.80/scripts/leaking_addresses.pl
linux-6.1.80/scripts/link_vmlinux.sh
linux-6.1.80/scripts/makefst
linux-6.1.80/scripts/markup_oops.pl
linux-6.1.80/scripts/mkn-tool-version.sh
linux-6.1.80/scripts/mkcompile_h
linux-6.1.80/scripts/mksynap
linux-6.1.80/scripts/mkuboot.sh
linux-6.1.80/scripts/mod/
linux-6.1.80/scripts/mod/.gitignore
linux-6.1.80/scripts/mod/Makefile
linux-6.1.80/scripts/mod/devicetable-offsets.c
linux-6.1.80/scripts/mod/empty.c
linux-6.1.80/scripts/mod/file2alias.c
linux-6.1.80/scripts/mod/list.h
linux-6.1.80/scripts/mod/nk_elfconfig.c
linux-6.1.80/scripts/mod/modpost.c
linux-6.1.80/scripts/mod/modpost.h
linux-6.1.80/scripts/mod/sunversion.c
linux-6.1.80/scripts/module.lds.S
linux-6.1.80/scripts/modules-check.sh
linux-6.1.80/scripts/nsdeps
linux-6.1.80/scripts/objdiff
linux-6.1.80/scripts/objdump-func
linux-6.1.80/scripts/package/
linux-6.1.80/scripts/package/bulddb
linux-6.1.80/scripts/package/buldtar
linux-6.1.80/scripts/package/mkdebian
linux-6.1.80/scripts/package/mkspec
linux-6.1.80/scripts/package/snapcraft.template
linux-6.1.80/scripts/pahote-flags.sh

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ ls
arch          COPYING      drivers      fs           gcc-uring   kernel       Makefile     modules.order  rust         sound        virt          vmlinux.map
bin           CREDITS     fs           gcc-uring   idb         lib          module.lds.S  Module.symvers  samples      System.map   vmlinux.map  vmlinux.o
boot-lzma    build       include      kbuild      kconfig     kselftest   modules.builtin  modules.builtin.modinfo  README      scripts      security     var           vmlinux.a
Documentation  kallsyms   kallsyms    Kconfig     MAINTAINERS  modules.builtin.modinfo  README          scripts        security     var           vmlinux-gdb.py
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ nano Makefile

```

Figure 2: Extracting the downloaded kernel

3 Makefile Modification (Kernel Branding)

To prove that this kernel was custom-built for this assignment, I modified the main Makefile of the kernel source. I appended my roll number to the EXTRAVERSION variable so that it reflects in the system's kernel release name.

- **Modification:** Changed the line to EXTRAVERSION = -K240707

```

GNU nano 7.2 Makefile
# SPDX-License-Identifier: GPL-2.0
VERSION = 6
PATCHLEVEL = 1
SUBLEVEL = 80
EXTRAVERSION = -K240707
NAME = Curry Ramen

# "DOCUMENTATION"
# To see a list of typical targets execute "make help"
# More info can be located in ./README
# Comments in this file are targeted only to the developer, do not
# expect to learn how to build the kernel reading this file.

$(if $(filter __%, $(MAKECMDRGS)), \
$(error targets prefixed with '__' are only for internal use))

# That's our default target when none is given on the command line
PHONY += _all
_all:
    @:

# We are using a recursive build, so we need to do a little thinking
# to get the ordering right.
#
# Most importantly: sub-Makefiles should only ever modify files in
# their own directory. If in some directory we have a dependency on
# a file in another dir (which doesn't happen often, but it's often
# unavoidable when linking the built-in targets which finally
# turn into vmlinux), we will call a sub-make in that other dir, and

```

Figure 3: Modifying the EXTRAVERSION variable in the main Makefile

4 System Call Implementation

Implementing the custom system call involved modifying the kernel source code in four distinct areas:

4.1 Creating the System Call Logic

I created a new C file named `hello_syscall.c` inside the `kernel/` directory. This file contains the actual logic to print my custom message to the kernel log.

```
1 #include <linux/kernel.h>
2 #include <linux/syscalls.h>
3
4 SYSCALL_DEFINE0(hello) {
5     printk("Hello World! system call for roll number K240707\n");
6     return 0;
7 }
```

4.2 Linking the System Call

To ensure the compiler includes my new C file during the build process, I appended the object file to the `kernel/Makefile`.

```
1 obj-y += hello_syscall.o
```

4.3 Updating the System Call Table

I registered my system call in the 64-bit system call table (`arch/x86/entry/syscalls/syscall_64.tbl`) by assigning it the specific system call number 451.

```
1 451      common      hello      sys_hello
```

4.4 Updating the Header File

Finally, I declared the function prototype in the main system calls header file (`include/linux/syscalls`) just before the `#endif` directive.

```
1 asmlinkage long sys_hello(void);
```

```
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80/kernel
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80/kernel$ cd kernel
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80/kernel$ nano hello_syscall.c
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80/kernel$

GNU nano 7.2 hello_syscall.c
#include <linux/kernel.h>
#include <linux/syscalls.h>

SYSCALL_DEFINE1(hello) {
    printk("Hello World! system call for roll number K240707\n");
    return 0;
}

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80/kernel$ nano Makefile
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80/kernel$

GNU nano 7.2 Makefile *
CFLAGS_stackleak.o := $(CFLAGS) -fstackleak=initial
obj-$(CONFIG_GCC_PLUGINS_STACKLEAK) += stackleak.o
KASAN_SANITIZE_stackleak.o := n
KCSAN_SANITIZE_stackleak.o := n
KCOV_INSTRUMENT_stackleak.o := n

obj-$(CONFIG_GCC_PLUGIN_TEST) += scf torture.o

$(obj)/config_data: $(obj)/config_data.gz
targets += config_data config_data.gz
$(obj)/config_data.gz: $(obj)/config_data FORCE
$(call if_changed,gzip)

filechk_cat: cat <=
$(obj)/config_data: $(CONFIG_CONFIG) FORCE
$(call filechk,cat)

$(obj)/kheaders.o: $(obj)/kheaders_data.tar.xz
quiet_cmd_genikh = CHK $(obj)/kheaders_data.tar.xz
cmd_genikh = $(CC) $(CFLAGS) $(CC) $(CFLAGS) /kernel/gen_kheaders.sh $@
$(obj)/kheaders_data.tar.xz: FORCE
$(call cmd,genikh)

clean-files += kheaders_data.tar.xz kheaders.nd5
obj-y += hello_syscall.o

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ nano arch/x86/entry/syscalls/syscall_64.tbl
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$

GNU nano 7.2 arch/x86/entry/syscalls/syscall_64.tbl *
429 common move_mount sys_move_mount
430 common fsopen sys_fsopen
431 common fsconfig sys_fsconfig
432 common fsmount sys_fsmount
433 common fspick sys_fspick
434 common pidfd_open sys_pidfd_open
435 common clone3 sys_clone3
436 common close_range sys_close_range
437 common openat2 sys_openat2
438 common pidfd_getfd sys_pidfd_getfd
439 common faccessat2 sys_faccessat2
440 common process_madvise sys_process_madvise
441 common epoll_pwait2 sys_epoll_pwait2
442 common mount_setattr sys_mount_setattr
443 common quotactl_fd sys_quotactl_fd
444 common landlock_create_ruleset sys_landlock_create_ruleset
445 common landlock_add_rule sys_landlock_add_rule
446 common landlock_restrict_self sys_landlock_restrict_self
447 common memfd_secret sys_memfd_secret
448 common process_release sys_process_release
449 common futex_waitv sys_futex_waitv
450 common set_mempolicy_home_node sys_set_mempolicy_home_node
451 common hello sys_hello

#
# Due to a historical design error, certain syscalls are numbered differently
# in x32 as compared to native x86_64. These syscalls have numbers 512-547.
# Do not add new syscalls to this range. Numbers 548 and above are available

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ nano include/linux/syscalls.h
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$
```

Figure 4: Modifying the system call table and headers (Part 1)

```

GNU nano 7.2 include/linux/syscalls.h
long __ksys_senget(int key, struct senbuf __user *tsops,
                  unsigned int nsops,
                  const struct __kernel_timespec __user *timeout);
long ksys_senget(key_t key, int nsens, int senflg);
long ksys_old_senctl(int semid, int sennum, int cnd, unsigned long arg);
long ksys_msgget(key_t key, int msgflg);
long ksys_old_msgctl(int msqid, int cnd, struct msqid_ds __user *buf);
long ksys_msgrcv(int msqid, struct msqbuf __user *msgp, size_t msgsz,
                 int msgtyp, int msgflg);
long ksys_msgsnd(int msqid, struct msqbuf __user *msgp, size_t msgsz,
                 int msgflg);
long ksys_shmget(key_t key, size_t size, int shmflg);
long ksys_shmdt(__user *shmaddr);
long ksys_old_shmctl(int shmid, int cnd, struct shm_id __user *buf);
long compat_ksys_senget(int semid, struct senbuf __user *tsens,
                        unsigned int nsops,
                        const struct old_timespec32 __user *timeout);
long __do_senget(int semid, struct senbuf *tsens, unsigned int nsops,
                 const struct timespec64 *timeout,
                 struct ipc_namespace *ns);

int __sys_getsockopt(int fd, int level, int optname, char __user *optval,
                    int __user *optlen);
int __sys_setsockopt(int fd, int level, int optname, char __user *optval,
                    int __user *optlen);
asmlinkage long sys_hello(void);
#endif

```

Figure 5: Modifying the system call table and headers (Part 2)

5 Kernel Configuration

Instead of writing a configuration from scratch, I copied the configuration file of my currently running system. This guarantees that the new kernel will have all the necessary drivers for my specific hardware. I then updated the configuration to the default settings for the new kernel version.

```

1 cp -v /boot/config-$(uname -r) .config
2 make olddefconfig

```

```

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ cp -v /boot/config-6.17.0-14-generic .config
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ make olddefconfig
.config:811:warning: symbol value 'm' invalid for HAVE_KVM_IRQ_BYPASS
.config:2848:warning: symbol value 'm' invalid for NVME_AUTH
.config:4892:warning: symbol value 'm' invalid for SERIAL_SC16IS7XX_I2C
.config:4893:warning: symbol value 'm' invalid for SERIAL_SC16IS7XX_SPI
.config:10176:warning: symbol value 'm' invalid for CROS_EC_PROTO
.config:11452:warning: symbol value 'm' invalid for ANDROID_BINDER_IPC
.config:11453:warning: symbol value 'm' invalid for ANDROID_BINDERFS
.config:11796:warning: override: SQUASHFS_DECOMP_MULTI changes choice state
.config:11797:warning: override: SQUASHFS_DECOMP_MULTI_PERCPU changes choice state
configuration written to .config
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$

```

Figure 6: Copying and updating the kernel configuration

6 Bypassing Security Certificates

A common issue when compiling on Ubuntu is the build failing due to missing Canonical security keys. To bypass this, I utilized the kernel's configuration script to disable the requirement for trusted and revocation keys.

```

1 scripts/config --disable SYSTEM_TRUSTED_KEYS
2 scripts/config --disable SYSTEM_REVOCATION_KEYS

```

```

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ scripts/config --disable SYSTEM_TRUSTED_KEYS
scripts/config --disable SYSTEM_REVOCATION_KEYS

```

Figure 7: Disabling system trusted and revocation keys

7 Final Compilation

With the environment fully configured, the system call implemented, and the keys by-passed, I initiated the final compilation. I used the `-j$(nproc)` flag to utilize all available CPU cores for a faster build.

```
1 make -j$(nproc)
```

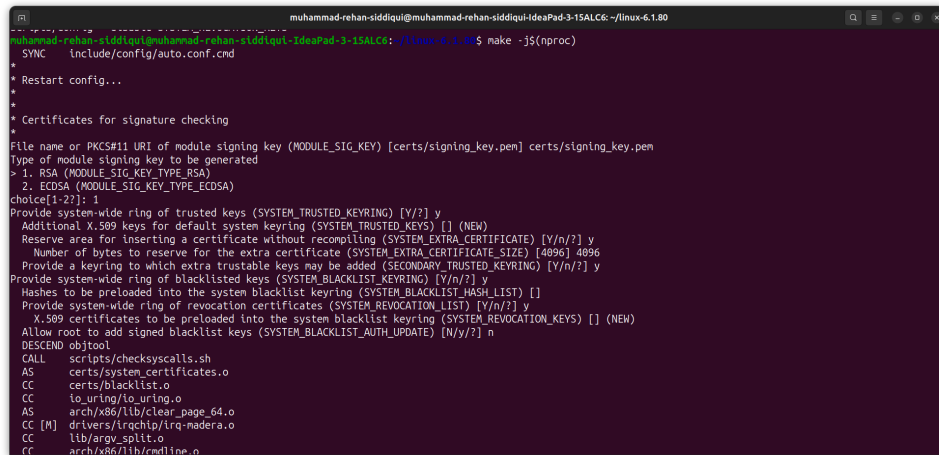


Figure 8: Executing the final compilation utilizing all CPU cores

8 Kernel Installation and System Reboot

After a successful compilation, the final phase is to install the compiled modules and the kernel image into the system's `/boot` directory. Finally, the GRUB bootloader must be updated to recognize the newly installed custom kernel before rebooting the system.

```
1 sudo make modules_install
2 sudo make install
3 sudo update-grub
4 sudo reboot
```

```

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ sudo make modules_install
[sudo] password for muhammad-rehan-siddiqui:
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/aegis128-aesni.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/aegis128-aesni.ko
ZSTD /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/aegis128-aesni.ko.zst
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/aesni-intel.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/aesni-intel.ko
ZSTD /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/aesni-intel.ko.zst
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/aria-aesni-avx-x86_64.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/aria-aesni-avx-x86_64.ko
ZSTD /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/aria-aesni-avx-x86_64.ko.zst
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/blowfish-x86_64.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/blowfish-x86_64.ko
ZSTD /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/blowfish-x86_64.ko.zst
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/camellia-aesni-avx-x86_64.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/camellia-aesni-avx-x86_64.ko
ZSTD /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/camellia-aesni-avx-x86_64.ko.zst
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/camellia-aesni-avx2.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/camellia-aesni-avx2.ko
ZSTD /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/camellia-aesni-avx2.ko.zst
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/camellia-x86_64.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/camellia-x86_64.ko
ZSTD /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/camellia-x86_64.ko.zst
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/cast5-avx-x86_64.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/cast5-avx-x86_64.ko
ZSTD /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/cast5-avx-x86_64.ko.zst
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/cast6-avx-x86_64.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/cast6-avx-x86_64.ko
ZSTD /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/cast6-avx-x86_64.ko.zst
INSTALL /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/crc32c-intel.ko
SIGN /lib/modules/6.1.80-K240707/kernel/arch/x86/crypto/crc32c-intel.ko
done

muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/linux-6.1.80$ sudo update-grub
Sourcing file /etc/default/grub
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.17.0-14-generic
Found initrd image: /boot/initrd.img-6.17.0-14-generic
Found linux image: /boot/vmlinuz-6.1.80-K240707
Found initrd image: /boot/initrd.img-6.1.80-K240707
Found initrd image: /boot/initrd.img-6.1.80-K240707
Found initrd image: /boot/initrd.img-6.1.80-K240707
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done

```

Figure 9: Installing modules, updating GRUB, and rebooting into the new kernel

9 Verification

After rebooting, the first step is to verify that the system is successfully running the newly compiled custom kernel. Then, a user-space C program is created to invoke the new system call. Finally, the kernel logs are checked to confirm the system call executed successfully and printed the correct message.

9.1 Verifying the Kernel Version

```

1 uname -r
2 # Expected Output: 6.1.80-K240707

```

9.2 Writing the Test Program (task.c)

I created a simple C program to test the newly added system call (number 451).

```

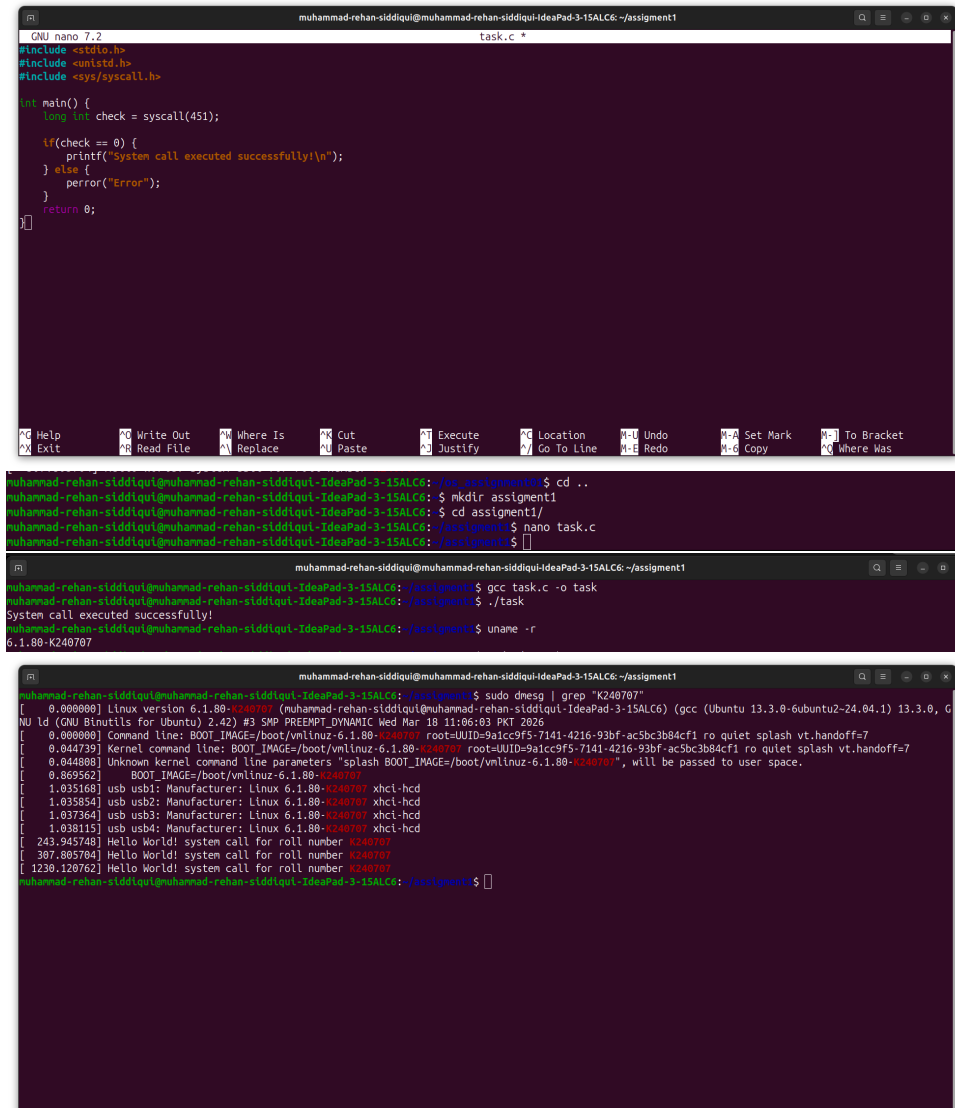
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/syscall.h>
4 int main() {
5     long int check = syscall(451);
6     if(check == 0) {
7         printf("System call executed successfully!\n");
8     } else {
9         perror("Error");
10    }
11    return 0;
12 }

```


9.3 Executing and Checking Logs

I compiled and ran the test program, then checked the `dmesg` logs to verify that the kernel printed my roll number.

```
1 gcc task.c -o task
2 ./task
3 sudo dmesg | grep "K240707"
```



The figure consists of three terminal screenshots. The first screenshot shows a nano editor window with the following C code:

```
GNU nano 7.2 task.c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

int main() {
    long int check = syscall(451);

    if(check == 0) {
        printf("System call executed successfully!\n");
    } else {
        perror("Error");
    }
    return 0;
}
```

The second screenshot shows the terminal output after running the program:

```
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/assignment1
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: $ cd ..
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: $ mkdir assignment1
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: $ cd assignment1/
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/assignment1$ nano task.c
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/assignment1$
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/assignment1$ gcc task.c -o task
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/assignment1$ ./task
System call executed successfully!
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/assignment1$ uname -r
6.1.80-K240707
```

The third screenshot shows the output of the `dmesg` command filtered by the roll number:

```
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/assignment1$ sudo dmesg | grep "K240707"
[ 0.000000] Linux version 6.1.80-K240707 (muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6) (gcc (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0, G
NU ld (GNU Binutils for Ubuntu) 2.42) #3 SMP PREEMPT_DYNAMIC Wed Mar 18 11:06:03 PKT 2026
[ 0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.1.80-K240707 root=UUID=9a1cc9f5-7141-4216-93bf-ac5bc3b84cf1 ro quiet splash vt.handoff=7
[ 0.044739] Kernel command line: BOOT_IMAGE=/boot/vmlinuz-6.1.80-K240707 root=UUID=9a1cc9f5-7141-4216-93bf-ac5bc3b84cf1 ro quiet splash vt.handoff=7
[ 0.044888] Unknown kernel command line parameters "splash BOOT_IMAGE=/boot/vmlinuz-6.1.80-K240707", will be passed to user space.
[ 0.069562]   BOOT_IMAGE=/boot/vmlinuz-6.1.80-K240707
[ 1.033160] usb1: Manufacturer: Linux 6.1.80-K240707 xhci-hcd
[ 1.035854] usb2: Manufacturer: Linux 6.1.80-K240707 xhci-hcd
[ 1.037364] usb3: Manufacturer: Linux 6.1.80-K240707 xhci-hcd
[ 1.038115] usb4: Manufacturer: Linux 6.1.80-K240707 xhci-hcd
[ 243.945748] Hello World! system call for roll number K240707
[ 307.805784] Hello World! system call for roll number K240707
[ 1230.120762] Hello World! system call for roll number K240707
muhammad-rehan-siddiqui@muhammad-rehan-siddiqui-IdeaPad-3-15ALC6: ~/assignment1$
```

Figure 10: Verifying the custom kernel version and system call execution in `dmesg`